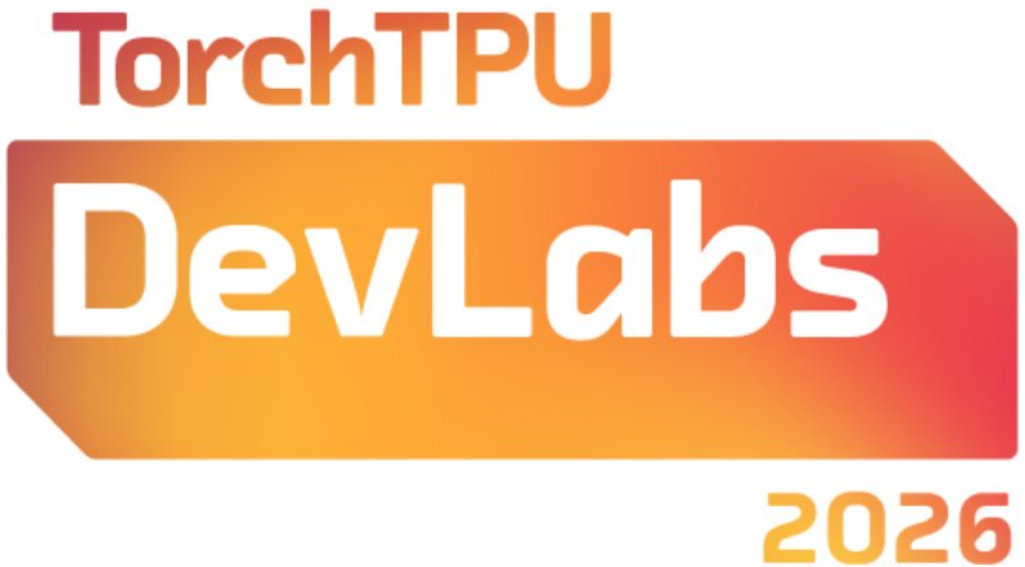
Inference with TorchTPU

TorchTPU DevLabs · Nov 9, 2026



Rob Mulla, Google
Developer Relations Engineer

Running inference on TPUs with PyTorch

1

TorchTPU: the TPU is a PyTorch device

the device line

three modes

demo 1

2

Inference 101: what decides serving speed

the KV cache

prefill and decode

capacity and bandwidth

3

vLLM and SGLang on TorchTPU

install and serve

demo 2: prefix caching

demo 3: continuous batching

`github.com/RobMulla/torchtpu-inference-workshop`

The code of every demo, these slides, and three Colab notebooks to run it yourself.

TorchTPU makes the TPU a PyTorch device

PyTorch
your model, your code

→

TorchTPU
registers the `tpu` device

→

Cloud TPU
one chip, or thousands

the whole change

```
model = model.to("tpu")
model = torch.compile(model, backend="tpu")  # optional
```

Eager



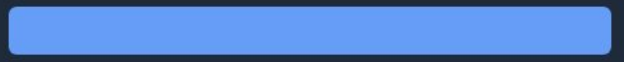
One op at a time

DeferAndFuse



Ops fused in blocks

torch.compile



One program

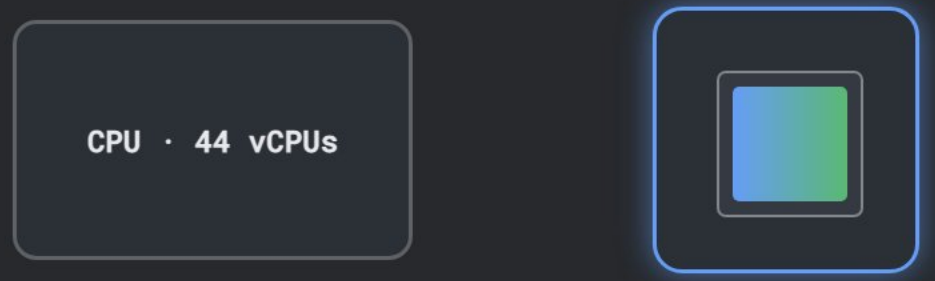
Demo 1: one script, three ways

`example1_eager.py` · a small model, forward passes in a loop · `device = "tpu"`

PyTorch · eager · DeferAndFuse · `torch.compile(backend="tpu")`

TorchTPU

Cloud TPU v6e-1 · one VM, one Trillium chip



1 Eager

```
$ python example1_eager.py
```

2 DeferAndFuse · the same file

```
$ TORCH_TPU_DEFER_AND_FUSE=1 \
    python example1_eager.py
```

3 Compiled · one line added

```
$ python example2_compile.py
```

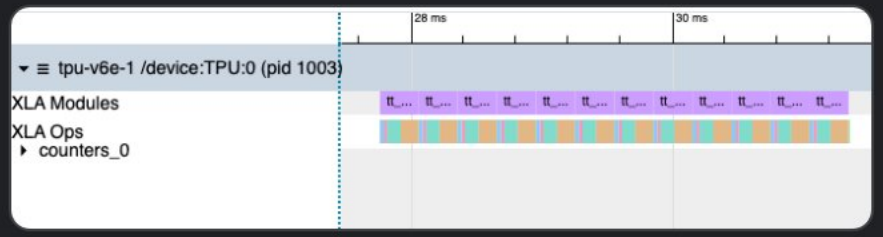
● LIVE · switching to the terminal

Profile results: one step of each

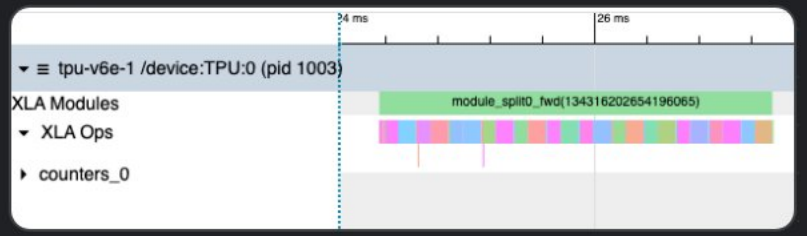
Eager · one op at a time



DeferAndFuse · ops fused in blocks

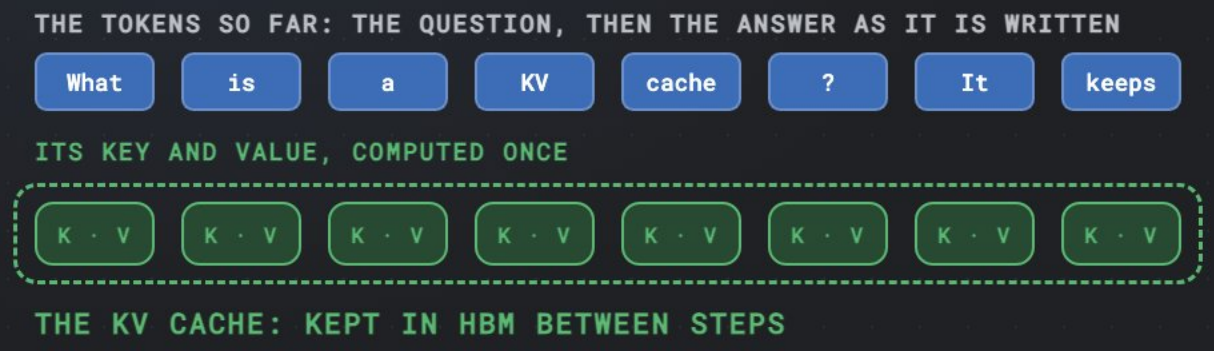


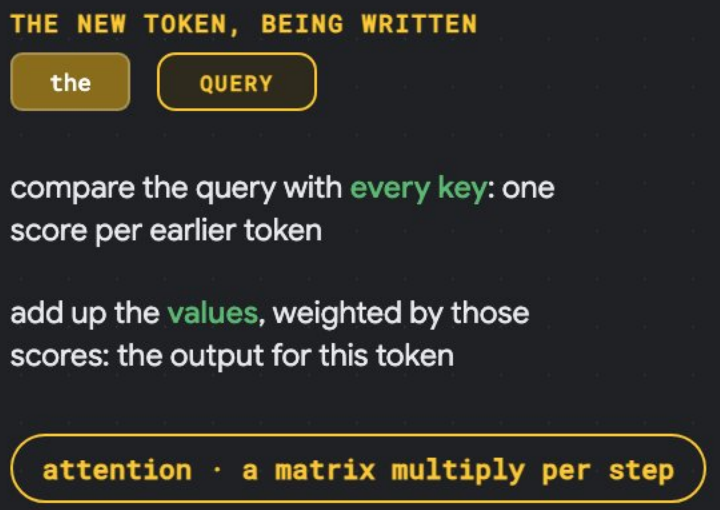
Compiled · one program



Debug in eager · speed up with one variable · ship compiled.

The KV cache: what makes serving an LLM different





KV cache

the keys and values of every token so far

Earlier tokens' keys and values never change, so keep them. Each step computes only the new token's and reads the rest.

Serving a model is two phases

Prefill

THE PROMPT



MATRIX UNITS

The whole prompt goes through the model at once.

Heavy when prompts are long: summarizing a document, a codebase in context.

compute-bound

Decode

HBM: WEIGHTS AND KV CACHE

THE ANSWER



One token a step; every step re-reads the weights and the cache.

Heavy when answers are long: chat, code generation, reasoning, agents.

memory-bound

Long answers are mostly decode, so serving is a memory problem.

Inference hardware: two numbers decide most of serving

Serving keeps the weights and the KV cache in HBM and reads them for every token. So, two numbers on the chip:

Capacity

A 32B MODEL IN BFLOAT16 ON FOUR TRILLIUM CHIPS



- Does the model fit?
- What is left for the KV cache?
- Too big for one chip: split it over four

`--tensor-parallel-size 4`

Bandwidth

ONE USER, ONE STEP



- Every token: read the weights and the cache
- Faster reads, faster tokens

`bytes per token`

Two questions to ask of any chip: does it fit, and how fast can you read it?

HBM per chip, by TPU generation



Per chip, one chip per generation. v5e, Trillium, Ironwood: the Cloud TPU documentation's specification tables. * TPU 8i: announced, "built for large-scale inference and reinforcement learning"; its figures from the Google Cloud blog, TPU 8t and TPU 8i technical deep dive (Apr 22, 2026).

Capacity is what fits; bandwidth is how fast the tokens come out.

TorchTPU runs in the serving engines you already use



vLLM · vllm-torchtpu
built with Inferact, founded by the creators of vLLM



SGLang · sglang-torchtpu
built with RadixArk, founded by the creators of SGLang

install, then serve

```
$ pip install vllm-torchtpu

$ vllm serve Qwen/Qwen3-32B \
    --tensor-parallel-size 4

# no TPU flag: the plugin registers itself
```

install, then serve

```
$ pip install sglang-*.whl sglang_tpu-*.whl
# the two release wheels of sglang-torchtpu
$ python -m sglang.launch_server \
    --model-path Qwen/Qwen3-32B --device tpu \
    --tp-size 4 --enable-torch-compile
```

Same flags, same API.

TPU Inference Stack

AGENTS AND APPLICATIONS	your chat, your coding tool, your agent sandbox	
ORCHESTRATION AND CONTROL	GKE · llm-d · Ray	TOMORROW, 1 PM: NEWTON
INFERENCE ENGINES	vLLM · SGLang · vllm-torchtpu · sglang-torchtpu	DEMOS 2 AND 3
KV CACHE MANAGEMENT	TPU Sync · Mooncake Store · LMCache · HiCache	
FRAMEWORK, COMPILER, KERNELS	PyTorch · TorchTPU · Pallas · OpenXLA	DEMO 1
TPU · STORAGE · NETWORK	Trillium, Ironwood · DRAM, Lustre · ICI, DCN	

What the engines handle for you

In both engines, supported today or being added.

Agents and multi-turn

- prefix caching demo 2
- the KV cache kept in host memory
- tool calling

Many users at once

- continuous batching demo 3
- speculative decoding
- structured output

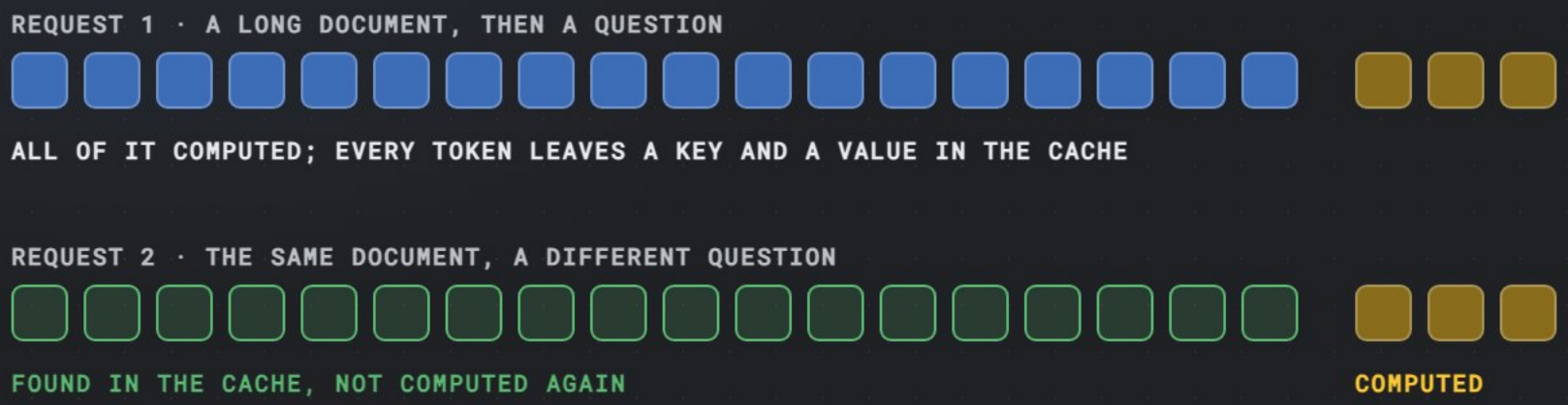
Big models, long contexts

- quantized checkpoints
- a compact FP8 KV cache
- long prompts split across chips

Many chips, many hosts

- every kind of parallelism
- prefill and decode on separate hosts
- multimodal input

Prefix caching



KV CACHE

kept after the request, keyed by the tokens

The cache is kept in pages. A request that starts the same way points at them: a pointer, not a copy. On by default in both engines.

Demo 2: one long prompt, asked about twice

ask.py · one long prompt, two questions — `:30000/v1`

SGL Qwen3-32B · four chips · prefix caching on

TorchTPU

Cloud TPU v6e-4 · one VM, four Trillium chips



Qwen3-32B split four ways · --tp-size 4

1 The server, already running

```
python -m sglang.launch_server \
    --device tpu --tp-size 4
```

2 A long document, question one

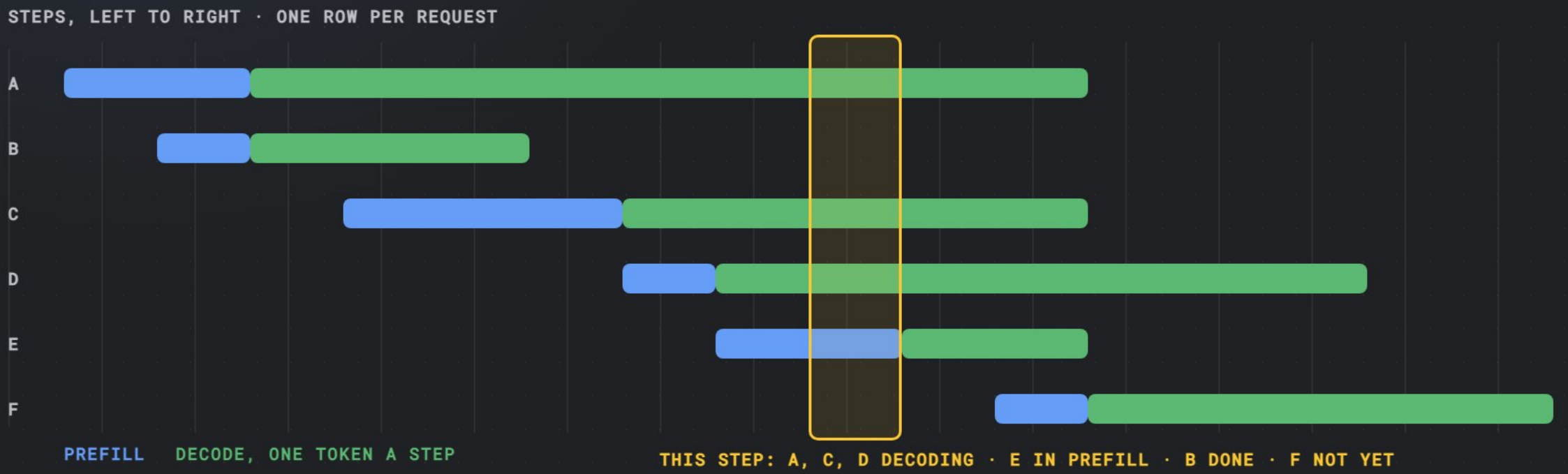
```
$ python ask.py
~11k tokens, nothing cached yet
```

3 The same document, question two

```
the same run asks again
watch: cached tokens, time to first token
```

● LIVE · switching to the terminal

Continuous batching: requests join and leave every step



The batch is re-formed every step: a request joins as soon as it arrives and leaves as soon as it finishes. One weight read serves everyone in the step.

`--max-num-seqs 128`

Demo 3: one request, then 64 at once

burst.py · 1 request, then 64 at once — :8000/v1

vLLM Qwen3-32B · four chips · continuous batching

TorchTPU

Cloud TPU v6e-4 · one VM, four Trillium chips



Qwen3-32B split four ways · --tensor-parallel-size 4

1 **The server, already running**

```
vllm serve Qwen/Qwen3-32B \
    --tensor-parallel-size 4
```

2 **One request, streamed**

```
$ python burst.py 1
one user, streaming
```

3 **Sixty-four at once**

```
$ python burst.py 64
watch: all four chips, tokens per second
```

● LIVE · switching to the terminal

Three things to take home

1. TorchTPU: the TPU is a PyTorch device. Set the device and run.

2. Serving speed is a memory story: the KV cache, capacity, bandwidth.

3. vLLM and SGLang run on TorchTPU the way you already use them.

Try it yourself



`github.com/RobMulla/torchtpu-inference-workshop`

- the code of the three demos
- three Colab notebooks: the same on one chip, a small model
- these slides

Questions now, or find me after. `The terminal stays open.`

Appendix: paged attention, and the ragged version TPUs use

requests

A: What | is | a | KV | cache | ? | It | keeps | past | keys

B: Name | three | TPUs

C: What | is | a | KV | page | ?

C starts with A's text. Its first page is A's page, shared, not copied. Only its two new tokens are computed.

Paged: fixed pages and a table per request, like an operating system's virtual memory. Nothing reserved, nothing wasted.

Ragged, on a TPU: fixed-shape programs, so the mixed batch is one kernel over the pages, in fixed tiles, no padding.

Shared: a prompt that starts like an earlier one points at its pages instead of recomputing them.

Simplified: one entry per token stands for one per layer; six tokens for thousands; pages of four for the engines' pages of 128.



this step

matrix units busy

HBM read per token